How to Store a Beacon and Its Short-Range Wireless Agents in Memory

Author Name
Department, Institution
author@example.com

I. Introduction

We have one beacon and N small wireless devices, called agents, placed around it. We number the beacon 0 and the agents 1 to N, so there are $n = N + 1$ nodes in total. Two nodes are *linked* if they are closer than the radio range R, which turns the network into a graph. Throughout this paper we use one small example with $n = 5$ nodes. Agents 1, 2 and 3 can hear the beacon directly. Agent 4 is too far from the beacon, but it can hear agent 3. Agents 1 and 2 can hear each other, and so can agents 2 and 3. The question is how to write this graph into memory.

II. Dense Matrix Storage

The simplest method is a table of 0s and 1s called the adjacency matrix $\mathbf{A}$, where $A_{ij} = 1$ if nodes i and j are linked and 0 otherwise. For our example, with the nodes in the order $(B, 1, 2, 3, 4)$:

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix} \tag{1}$$

The first row is the beacon's row, and it shows that agents 1, 2 and 3 hear the beacon. Since a link works in both directions, the matrix is a mirror image of itself ($\mathbf{A} = \mathbf{A}^{\mathsf{T}}$), so we only need to keep the part above the diagonal. Read row by row, that part is the bit string $1110\,|\,100\,|\,10\,|\,1$. The full matrix needs $n^2 = 25$ bits, and the upper part needs only $n(n-1)/2 = 10$ bits.

III. Sparse Storage

When devices are spread over a large area, each one hears only a few neighbours, so almost every entry of the matrix is 0. A cheaper way is to store only the neighbours of each node. This is the adjacency list:

$$\begin{array}{ll} 0:\ 1, 2, 3 & 3:\ 0, 2, 4 \\ 1:\ 0, 2 & 4:\ 3 \\ 2:\ 0, 1, 3 & \end{array} \tag{2}$$

In memory, the same list is kept as two flat arrays, called compressed sparse row (CSR) form. The first array joins all the neighbour lists. The second says where each node's list starts:

$$\text{neighbours} = [\,1,2,3,\ 0,2,\ 0,1,3,\ 0,2,4,\ 3\,] \tag{3}$$

$$\text{start} = [\,0,\ 3,\ 5,\ 8,\ 11,\ 12\,] \tag{4}$$

The neighbours of node i are those between positions start$[i]$ and start$[i+1]-1$. With E links, each stored twice, and each number written with $\lceil \log_2 n \rceil$ bits, the list needs

$$S_{\text{list}} = 2E\,\lceil \log_2 n \rceil \text{ bits.} \tag{5}$$

Here $E = 6$, so $S_{\text{list}} = 12 \times 3 = 36$ bits, which is more than the matrix. The list wins only when the average number of neighbours satisfies $\bar{k} < n/\lceil \log_2 n \rceil$, which happens for large, spread-out networks.

IV. Extra Data and Recommendation

Each agent also needs its distance from the beacon in hops, stored as one small number per node. For our example this is

$$\mathbf{h} = [\,0,\ 1,\ 1,\ 1,\ 2\,], \tag{6}$$

which says that agents 1, 2 and 3 are one hop away and agent 4 is two hops away. We recommend the plain bit matrix for small networks, where it is both smaller and faster to look up. For large networks we recommend the two CSR arrays plus the hop array $\mathbf{h}$. In both cases, keep the beacon's row as its own bit mask, since it answers the most common question: who hears the beacon directly?